MODUL PRAKTIKUM

PEMROGRAMAN PYTHON

Logika Aplikasi:

Variabel, I/O, Kondisional, Perulangan,

dan Validasi Aturan Bisnis

Program Studi Sistem Informasi

Mata Kuliah Pemrograman Python

DAFTAR ISI

Bab 1: Pengantar Logika Aplikasi dan Aturan Bisnis 1

1.1 Ap itu Aturan Bisnis? 1

1.2 Sintaks Dasar Python untuk Logika 2

Bab 2: Variabel dan Tipe Data untuk Logika Bisnis 3

2.1 Variabel dan Konvensi Penamaan 3

2.2 Tipe Data untuk Entitas Bisnis 4

2.3 Operator Perbandingan dan Logika 5

Bab 3: Input dan Output (I/O) 6

3.1 Fungsi input() dan Konversi Tipe 6

3.2 Fungsi print() dan Formatting Output 7

3.3 Studi Kasus: Form Input Data Mahasiswa 8

Bab 4: Struktur Kondisional (if-elif-else) 9

4.1 Percabangan Dasar: if dan if-else 9

4.2 Percabangan Berantai: if-elif-else 10

4.3 Nested If: Percabangan Bersarang 11

4.4 Operator Ternary (Conditional Expression) 12

Bab 5: Struktur Perulangan (for dan while) 12

5.1 For Loop: Iterasi over Sequence 13

5.2 For Loop dengan range() 13

5.3 While Loop: Perulangan Berdasarkan Kondisi 14

5.4 Break, Continue, dan Else pada Loop 15

Bab 6: Validasi Input dan Aturan Bisnis 16

6.1 Validasi Tipe dan Format 16

6.2 Validasi dengan Loop: Input Sampai Valid 17

6.3 Validasi Aturan Bisnis Komposit 18

Bab 7: Studi Kasus Integratif 19

7.1 Studi Kasus 1: Sistem Status Mahasiswa 19

7.2 Studi Kasus 2: Diskon Biaya Kuliah 20

7.3 Studi Kasus 3: Sistem Peringatan Stok 21

Rangkuman 23

Latihan Soal 24

A. Soal Teori 24

B. Soal Praktik 24

Bab 1: Pengantar Logika Aplikasi dan Aturan Bisnis

Logika aplikasi merupakan inti dari setiap program komputer. Setiap perangkat lunak, tidak terkecuali sistem informasi, dibangun di atas serangkaian aturan bisnis (business rules) yang menentukan bagaimana data diproses, keputusan dibuat, dan output dihasilkan. Dalam konteks Sistem Informasi, aturan bisnis ini mencakup segala hal mulai dari penentuan status mahasiswa berdasarkan IPK, perhitungan diskon biaya kuliah berdasarkan kriteria tertentu, hingga peringatan stok barang yang mendekati habis. Kemampuan mengimplementasikan aturan bisnis ke dalam kode program merupakan kompetensi fundamental yang harus dikuasai oleh setiap pengembang sistem informasi.

Python sebagai bahasa pemrograman menyediakan sintaks yang jernih dan ekspresif untuk mengimplementasikan logika aplikasi. Struktur kontrol alur (flow control) seperti percabangan (if-elif-else) dan perulangan (for, while) memungkinkan pengembang mengekspresikan keputusan bisnis secara langsung dalam kode. Ditambah dengan mekanisme input/output yang fleksibel dan kemampuan validasi data, Python menjadi alat yang sangat efektif untuk menerjemahkan spesifikasi bisnis menjadi program yang fungsional. Modul praktikum ini akan membimbing mahasiswa memahami dan mengimplementasikan berbagai aturan bisnis menggunakan konstruksi dasar Python.

1.1 Ap itu Aturan Bisnis?

Aturan bisnis adalah pernyataan logis yang mendefinisikan atau membatasi beberapa aspek dari operasi bisnis. Aturan ini mengontrol bagaimana organisasi menjalankan aktivitasnya, membuat keputusan, dan memproses informasi. Dalam pengembangan sistem informasi, aturan bisnis menjadi fondasi yang menentukan perilaku aplikasi. Sebagai contoh, sebuah universitas memiliki aturan bisnis yang menyatakan bahwa mahasiswa dinyatakan aktif jika IPK-nya di atas 2.0 dan memenuhi batas SKS minimum per semester. Aturan lainnya mungkin menentukan bahwa biaya kuliah mendapat diskon 25% jika mahasiswa merupakan anak karyawan internal atau memiliki prestasi akademik tertentu.

Komponen Aturan Bisnis	Deskripsi	Contoh pada SI Mahasiswa
Kondisi (Condition)	Syarat yang harus dipenuhi	IPK >= 2.0 DAN SKS >= 18
Aksi (Action)	Yang dilakukan jika kondisi benar	Status = Aktif
Pengecualian (Exception)	Kondisi khusus yang mengubah aksi	Cuti akademik mengubah status
Prioritas (Priority)	Urutan evaluasi jika ada konflik	Cek cuti sebelum cek IPK
Data Referensi	Data pendukung penentuan	Tabel batas IPK per prodi

> **i Aturan Bisnis vs Kode Program**
>
> Aturan bisnis ditulis dalam bahasa manusia dan sering berubah seiring kebijakan organisasi. Tugas pengembang adalah menerjemahkan aturan ini menjadi kode yang terstruktur, mudah dipelihara, dan mudah diadaptasi saat aturan bisnis berubah. Pemisahan logika bisnis dari tampilan (separation of concerns) merupakan praktik terbaik yang akan dibahas lebih lanjut.

1.2 Sintaks Dasar Python untuk Logika

Python menyediakan beberapa konstruksi sintaks fundamental yang menjadi building block untuk mengimplementasikan aturan bisnis. Variabel menyimpan data dan state, operator perbandingan dan logika mengekspresikan kondisi, struktur kondisional menangani percabangan keputusan, dan perulangan memproses data secara iteratif. Kombinasi keempat konstruksi ini memungkinkan implementasi aturan bisnis dari yang paling sederhana hingga yang cukup kompleks. Berikut gambaran singkat konstruksi yang akan dipelajari dalam modul ini.

```
# Variabel - menyimpan data aturan bisnis
batas_ipk = 2.0
batas_sks = 18
status_cuti = False

# Operator perbandingan dan logika
ipk = 3.5
sks = 20
layak_aktif = (ipk >= batas_ipk) and (sks >= batas_sks) and (not status_cuti)
```

```
# Struktur kondisional
if layak_aktif:
    status = "Aktif"
elif status_cuti:
    status = "Cuti"
else:
    status = "Tidak Aktif"

# Perulangan - memproses banyak data
daftar_mahasiswa = [{"nim": "001", "ipk": 3.5}, {"nim": "002", "ipk": 1.8}]
for mhs in daftar_mahasiswa:
    if mhs["ipk"] >= batas_ipk:
        print(f"{mhs['nim']} : Aktif")
    else:
        print(f"{mhs['nim']} : Tidak Aktif")
```

Bab 2: Variabel dan Tipe Data untuk Logika Bisnis

Variabel merupakan wadah penyimpanan data yang digunakan oleh program untuk merepresentasikan entitas dan state dalam aturan bisnis. Pemilihan tipe data yang tepat sangat penting karena menentukan operasi apa yang dapat dilakukan terhadap data dan bagaimana data tersebut dibandingkan. Dalam konteks Sistem Informasi, data mahasiswa direpresentasikan dengan berbagai tipe: string untuk NIM dan nama, float untuk IPK dan biaya, integer untuk SKS dan semester, boolean untuk status aktif atau cuti, serta list dan dictionary untuk koleksi data. Pemahaman mendalam tentang tipe data dan operasinya merupakan prasyarat sebelum mengimplementasikan logika bisnis yang lebih kompleks.

2.1 Variabel dan Konvensi Penamaan

Dalam Python, variabel dibuat secara dinamis melalui assignment tanpa memerlukan deklarasi tipe eksplisit. Meskipun fleksibel, penggunaan konvensi penamaan yang konsisten sangat penting untuk keterbacaan kode, terutama saat mengimplementasikan aturan bisnis yang kompleks. Snake case merupakan konvensi standar Python (PEP 8) untuk variabel dan fungsi, di mana kata dipisahkan dengan garis bawah dan seluruh huruf menggunakan lowercase. Nama variabel harus deskriptif dan mencerminkan entitas bisnis yang direpresentasikan, bukan singkatan yang ambigu atau nama generik seperti x, y, atau temp.

```
# Konvensi penamaan yang BAIK untuk aturan bisnis
batas_ipk_minimum = 2.0
persentase_diskon_prestasi = 25
status_mahasiswa_aktif = True
daftar_mahasiswa_cuti = []

# Penamaan yang KURANG BAIK - hindari
# x = 2.0          # terlalu generik
# d = 25            # ambigu: diskon apa?
# flag = True       # tidak deskriptif
# arr = []          # tipe jelas, tapi konteks bisnis?

# Konstanta - gunakan UPPER_SNAKE_CASE
IPK_MINIMUM_AKTIF = 2.0
SKS_MINIMUM_PER_SEMESTER = 18
BATAS_STOK_PERINGATAN = 10
DISKON_ANAK_KARYAWAN = 0.25
```

★ **Tips: Konstanta Aturan Bisnis**

Gunakan UPPER_SNAKE_CASE untuk nilai yang merupakan aturan bisnis tetap (konstanta), seperti batas IPK, persentase diskon, atau batas stok. Ini memudahkan identifikasi aturan bisnis dalam kode dan mempermudah perubahan saat kebijakan berubah karena semua referensi terkonsentrasi pada satu lokasi.

2.2 Tipe Data untuk Entitas Bisnis

Setiap entitas bisnis dalam Sistem Informasi memiliki atribut yang paling tepat direpresentasikan dengan tipe data tertentu. Pemilihan tipe data yang salah dapat menyebabkan bug halus, seperti perbandingan string yang tidak sesuai harapan atau presisi hilang pada perhitungan keuangan. Tabel berikut merangkum tipe data yang umum digunakan beserta contoh penerapannya dalam konteks SI MahA Mahasiswa.

Tipe Data	Deskripsi	Contoh pada SI Mahasiswa	Catatan Penting
str	Teks / string	NIM: "2021001", Nama: "Andi"	NIM jangan int: leading zero hilang
int	Bilangan bulat	SKS: 20, Semester: 5	Untuk hitungan / kuantitas
float	Bilangan desimal	IPK: 3.52, Biaya: 1500000.0	Hindari float untuk uang presisi
bool	Benar / Salah	is_aktif: True, is_cuti: False	Untuk status / flag

Tipe Data	Deskripsi	Contoh pada SI Mahasiswa	Catatan Penting
list	Koleksi terurut	[3.5, 2.8, 3.9] (daftar IPK)	Mutable, boleh duplikat
dict	Pasangan key-value	{"nim": "001", "ipk": 3.5}	Representasi record/entitas
tuple	Koleksi immutable	(2.0, 18) (batas IPK, SKS)	Untuk konstanta gabungan

```
# Representasi data mahasiswa dengan tipe data yang tepat
mahasiswa = {
    "nim": "2021001",        # str - NIM dengan leading zero
    "nama": "Andi Pratama",  # str
    "ipk": 3.52,             # float
    "sks_tempuh": 20,        # int
    "semester": 5,           # int
    "is_aktif": True,        # bool
    "is_cuti": False,        # bool
    "biaya_spp": 1509500.0,  # float (atau Decimal)
}

# Tipe data koleksi untuk banyak mahasiswa
daftar_nilai = [85, 90, 78, 92, 88]      # list: kumpulan nilai
batas_akademik = (2.0, 18)                # tuple: (ipk_min, sks_min)

# Akses dan modifikasi
print(f"Nama: {mahasiswa['nama']}, IPK: {mahasiswa['ipk']}")
mahasiswa["sks_tempuh"] += 3  # tambah SKS
```

2.3 Operator Perbandingan dan Logika

Operator perbandingan dan logika merupakan alat utama untuk mengekspresikan kondisi aturan bisnis. Operator perbandingan membandingkan dua nilai dan menghasilkan boolean (True/False), sementara operator logika menggabungkan beberapa kondisi menjadi ekspresi yang lebih kompleks. Dalam implementasi aturan bisnis, penggabungan kondisi dengan and, or, dan not sangat umum. Misalnya, status aktif mahasiswa memerlukan IPK di atas batas AND SKS mencukupi AND tidak dalam status cuti. Pent7 Urutan evaluasi (precedence) operator logika perlu dipahami untuk menghindari bug: not dievaluasi sebelum and, dan and sebelum or.

```
# Operator perbandingan
ipk = 3.5
print(ipk > 2.0)   # True  - lebih dari
print(ipk >= 3.0)  # True  - lebih dari atau sama
```

```
print(ipk < 4.0)   # True  - kurang dari
print(ipk <= 3.5)  # True  - kurang dari atau sama
print(ipk == 3.5)  # True  - sama dengan
print(ipk != 4.0)  # True  - tidak sama dengan

# Operator logika - menggabungkan kondisi bisnis
ipk = 3.5
sks = 20
is_cuti = False

# Aturan: aktif jika IPK >= 2.0 DAN SKS >= 18 DAN tidak cuti
syarat_ipk = ipk >= 2.0
syarat_sks = sks >= 18
status_aktif = syarat_ipk and syarat_sks and (not is_cuti)
print(f"Status aktif: {status_aktif}")  # True

# Aturan: diskon jika anak karyawan ATAU IPK >= 3.8
is_anak_karyawan = False
layak_diskon = is_anak_karyawan or (ipk >= 3.8)
print(f"Layak diskon: {layak_diskon}")  # False (3.5 < 3.8)
```

Bab 3: Input dan Output (I/O)

Interaksi dengan pengguna melalui input dan output merupakan aspek krusial dari setiap aplikasi. Dalam konteks Sistem Informasi, program perlu menerima data dari pengguna (misalnya data mahasiswa, nilai, atau jumlah stok), memproses data tersebut sesuai aturan bisnis, dan menampilkan hasil yang informatif. Python menyediakan fungsi input() untuk menerima data dari pengguna dan fungsi print() untuk menampilkan output, serta teknik formatting yang kaya untuk menyajikan data secara terstruktur dan mudah dibaca.

3.1 Fungsi input() dan Konversi Tipe

Fungsi input() selalu mengembalikan nilai bertipe string, sehingga konversi tipe (type casting) diperlukan ketika data yang diharapkan berupa angka atau tipe lainnya. Konversi ini rentan terhadap error jika pengguna memasukkan data yang tidak sesuai format, misalnya memasukkan teks saat angka diharapkan. Oleh karena itu, konversi tipe harus selalu dipasangkan dengan mekanisme validasi atau exception handling, yang akan dibahas secara mendalam pada Bab 6. Untuk saat ini, kita akan menggunakan konversi dasar dengan asumsi input yang valid.

```
# Input dasar - selalu menghasilkan str
nama = input("Masukkan nama mahasiswa: ")
print(f"Nama: {nama}, Tipe: {type(nome)}")

# Konversi tipe untuk data numerik
ipk_input = input("Masukkan IPK: ")
ipk = float(ipk_input)      # str -> float

sks_input = input("Masukkan SKS: ")
sks = int(sks_input)        # str -> int

# Input dengan konversi langsung (common pattern)
semester = int(input("Masukkan semester: "))
biaya = float(input("Masukkan biaya SPP: "))

# Konversi ke boolean - hati-hati!
# bool("False") menghasilkan True (string non-kosong)
# Gunakan perbandingan eksplisit
status = input("Aktif? (y/n): ")
is_aktif = status.lower() == "y"  # lebih aman
```

> ⚠ **Peringatan: input() Selalu String**
>
> Fungsi input() selalu mengembalikan string. Konversi ke int atau float akan gagal (ValueError) jika input bukan angka yang valid. Selalu lakukan validasi sebelum konversi, atau gunakan try-except untuk menangani input yang tidak valid.

3.2 Fungsi print() dan Formatting Output

Fungsi print() mendukung berbagai teknik formatting untuk menyajikan output aturan bisnis secara informatif dan profesional. F-string (formatted string literal) merupakan cara formatting yang paling direkomendasikan sejak Python 3.6 karena sintaksnya yang ringkas dan ekspresif. F-string mendukung format specifier untuk mengatur lebar field, jumlah desimal, alignment, dan bahkan ekspresi Python di dalam kurung kurawal. Untuk output tabel atau laporan, teknik formatting ini sangat berguna menyajikan data secara terstruktur.

```
# F-string formatting untuk output bisnis
nama = "Andi Pratama"
ipk = 3.521
biaya = 1509500
```

```
diskon = 0.25

# Format desimal
print(f"IPK: {ipk:.2f}")           # IPK: 3.52

# Format angka dengan separator ribuan
print(f"Biaya: Rp{biaya:,}")       # Biaya: Rp1,509,500

# Format persentase
print(f"Diskon: {diskon:.0%}")     # Diskon: 25%

# Alignment untuk tabel sederhana
print(f"{'Nama':<20} {'IPK':>5} {'Status':<10}")
print(f"{nama:<20} {ipk:>5.2f} {'Aktif':<10}")

# Output multi-line dengan f-string
ringkasan = f"""
=== Ringkasan Mahasiswa ===
NIM     : 2021001
Nama    : {nama}
IPK     : {ipk:.2f}
Biaya   : Rp{biaya:,}
Diskon  : {diskon:.0%}
"""
print(ringkasan)
```

3.3 Studi Kasus: Form Input Data Mahasiswa

Berikut adalah contoh lengkap program yang menerima input data mahasiswa dari pengguna, menyimpannya dalam variabel dan dictionary, lalu menampilkan ringkasan dengan format yang rapi. Program ini meng, merupakan cikal bakal modul input yang akan dikembangkan menjadi sistem informasi yang lebih lengkap pada bab-bab selanjutnya.

```
# studi_kasus_input_mahasiswa.py
"""Program input data mahasiswa sederhana."""

# Konstanta aturan bisnis
IPK_MINIMUM_AKTIF = 2.0
SKS_MINIMUM = 18

# Input data mahasiswa
print("=== Form Input Data Mahasiswa ===")
nim = input("NIM          : ")
nama = input("Nama         : ")
prodi = input("Program Studi: ")
ipk = float(input("IPK          : "))
sks = int(input("SKS Tempuh   : "))
biaya_spp = float(input("Biaya SPP    : Rp"))
```

```
# Hitung status berdasarkan aturan bisnis
if ipk >= IPK_MINIMUM_AKTIF and sks >= SKS_MINIMUM:
    status = "Aktif"
else:
    status = "Tidak Aktif"

# Tampilkan ringkasan
print("\n=== Ringkasan Data Mahasiswa ===")
print(f"NIM           : {nim}")
print(f"Nama          : {nama}")
print(f"Program Studi : {prodi}")
print(f"IPK           : {ipk:.2f}")
print(f"SKS Tempuh    : {sks}")
print(f"Biaya SPP     : Rp{biaya_spp:,.0f}")
print(f"Status        : {status}")
```

Bab 4: Struktur Kondisional (if-elif-else)

Struktur kondisional merupakan mekanisme utama untuk mengimplementasikan keputusan bisnis dalam program. Ketika aturan bisnis menyatakan bahwa suatu aksi dilakukan hanya jika kondisi tertentu dipenuhi, maka struktur if-elif-else adalah alat implementasinya. Dalam Sistem Informasi, hampir semua aturan bisnis melibatkan kondisi: penentuan status mahasiswa, kelayakan diskon, tingkat peringatan stok, kategori nilai, dan sebagainya. Kemampuan meng struktur percabangan dengan benar, termasuk nested if dan chained elif, merupakan keterampilan esensial yang menentukan kualitas kode implementasi aturan bisnis.

4.1 Percabangan Dasar: if dan if-else

Percabangan dasar dengan if mengeksekusi blok kode hanya jika kondisi bernilai True. Bentuk if-else menambahkan blok alternatif yang dijalankan jika kondisi False. Percabangan ini cocok untuk aturan bisnis biner, yaitu aturan yang memiliki tepat dua kemungkinan hasil, seperti mahasiswa aktif atau tidak aktif, layak diskon atau tidak layak, stok aman atau stok rendah. Perhatikan indentasi yang konsisten karena Python menggunakan indentasi untuk menentukan blok kode.

```
# Aturan bisnis: status mahasiswa berdasarkan IPK
ipk = float(input("Masukkan IPK: "))

if ipk >= 2.0:
    status = "Aktif"
```

```
    print("Mahasiswa berstatus Aktif.")
else:
    status = "Tidak Aktif"
    print("Mahasiswa berstatus Tidak Aktif.")

# Aturan bisnis: peringatan stok
stok = int(input("Masukkan jumlah stok: "))
batas = 10

if stok <= batas:
    print(f"PERINGATAN: Stok rendah ({stok} unit)!")
    print("Segera lakukan restock.")
else:
    print(f"Stok aman ({stok} unit).")
```

4.2 Percabangan Berantai: if-elif-else

Ketika aturan bisnis memiliki lebih dari dua kemungkinan hasil, percabangan berantai menggunakan if-elif-else digunakan. Struktur ini mengevaluasi kondisi secara berurutan dari atas ke bawah dan mengeksekusi blok pertama yang kondisinya True. Ini sangat sesuai untuk aturan bisnis yang mengkategorikan ke dalam beberapa level atau kategori, seperti penentuan grade nilai (A, B+, B, C, D, E), tingkat peringatan stok (aman, peringatan, kritis), atau kategori mahasiswa (cumlaude, sangat memuaskan, memuaskan, cukup). Urutan kondisi harus disusun dengan cermat agar tidak ada kasus yang terlewat atau salah dikategorikan.

```
# Aturan bisnis: Grade nilai akhir
nilai_akhir = float(input("Masukkan nilai akhir: "))

if nilai_akhir >= 85:
    grade = "A"
    predikat = "Sangat Memuaskan"
elif nilai_akhir >= 75:
    grade = "B+"
    predikat = "Memuaskan"
elif nilai_akhir >= 65:
    grade = "B"
    predikat = "Cukup Memuaskan"
elif nilai_akhir >= 55:
    grade = "C"
    predikat = "Cukup"
elif nilai_akhir >= 45:
    grade = "D"
    predikat = "Kurang"
else:
    grade = "E"
    predikat = "Tidak Lulus"
```

```
print(f"Grade: {grade} - {predikat}")
```

> **i Urutan Kondisi Penting**
>
> Dalam if-elif-else, kondisi dievaluasi berurutan dan hanya blok pertama yang True yang dijalankan. Urutkan dari kondisi paling spesifik/restrictif ke paling umum, atau dari nilai ambang tertinggi ke terendah (untuk rentang numerik). Kesalahan urutan menghasilkan kategori yang salah.

4.3 Nested If: Percabangan Bersarang

Percabangan bersarang (nested if) digunakan ketika keputusan bisnis bergantung pada kombinasi kondisi bertingkat. Misalnya, penentuan diskon biaya kuliah mungkin memerlukan pengecekan kategori mahasiswa terlebih dahulu, kemudian di dalam setiap kategori terdapat syarat tambahan yang berbeda. Meskipun nested if sangat berguna, kedalaman lebih dari tiga level membuat kode sulit dibaca dan sebaiknya direfaktor. Alternatif yang lebih bersih adalah menggabungkan kondisi dengan operator logika (and, or) atau menggunakan early return pattern.

```
# Aturan bisnis: Diskon biaya kuliah (nested if)
biaya_spp = 5000000

print("=== Cek Kelayakan Diskon ===")
is_anak_karyawan = input("Anak karyawan? (y/n): ").lower() == "y"
ipk = float(input("IPK: "))
semester = int(input("Semester: "))

diskon = 0.0

if is_anak_karyawan:
    # Diskon anak karyawan: 25% untuk semua semester
    diskon = 0.25
    kategori = "Anak Karyawan"
else:
    # Diskon prestasi: berdasarkan IPK dan semester
    if ipk >= 3.8 and semester >= 3:
        diskon = 0.20
        kategori = "Prestasi IPK Tinggi"
    elif ipk >= 3.5 and semester >= 2:
        diskon = 0.15
        kategori = "Prestasi IPK Baik"
    elif ipk >= 3.0:
        diskon = 0.10
```

```
        kategori = "IPK Memuaskan"
    else:
        diskon = 0.0
        kategori = "Tidak ada diskon"

potongan = biaya_spp * diskon
biaya_final = biaya_spp - potongan

print(f"\nKategori  : {kategori}")
print(f"Diskon    : {diskon:.0%}")
print(f"Potongan  : Rp{potongan:,.0f}")
print(f"Bayar     : Rp{biaya_final:,.0f}")
```

4.4 Operator Ternary (Conditional Expression)

Python mendukung conditional expression (operator ternary) yang memungkinkan penulisan percabangan sederhana dalam satu baris. Sintaksnya adalah nilai_jika_true if kondisi else nilai_jika_false. Bentuk ini sangat berguna untuk assignment variabel yang bergantung pada kondisi, membuat kode lebih ringkas tanpa mengorbankan keterbacaan. Namun, hindari nesting ternary karena sangat sulit dibaca.

```
# Operator ternary untuk assignment sederhana
ipk = 3.5
status = "Aktif" if ipk >= 2.0 else "Tidak Aktif"
print(f"Status: {status}")  # Aktif

# Ternary untuk peringatan stok
stok = 5
batas = 10
pesan = "STOK RENDAH!" if stok <= batas else "Stok aman"
print(pesan)  # STOK RENDAH!

# Ternary dalam f-string
is_lulus = True
print(f"Mahasiswa {'LULUS' if is_lulus else 'TIDAK LULUS'}")
```

Bab 5: Struktur Perulangan (for dan while)

Perulangan (looping) digunakan untuk memproses data secara berulang, yang merupakan operasi sangat umum dalam Sistem Informasi. Misalnya, menampilkan daftar seluruh mahasiswa, menghitung rata-rata nilai dari seluruh kelas, memeriksa stok semua barang, atau memproses transaksi satu per satu. Python menyediakan dua jenis perulangan: for loop untuk iterasi over sequence (list, tuple, string, range, dictionary) dan while loop untuk

perulangan yang dikontrol oleh kondisi. Pemilihan jenis loop yang tepat bergantung pada karakteristik data dan logika bisnis yang diimplementasikan.

5.1 For Loop: Iterasi over Sequence

For loop merupakan konstruksi perulangan yang paling sering digunakan dalam Python karena kemampuannya mengiterasi langsung over elemen sequence tanpa memerlukan indeks. Dalam konteks aturan bisnis, for loop digunakan untuk memproses setiap item dalam koleksi data: mengecek status setiap mahasiswa, menghitung diskon untuk setiap transaksi, atau memvalidasi setiap entri data. Fungsi range() sering dipasangkan dengan for loop ketika perulangan berdasarkan hitungan, sementara enumerate() menyediakan indeks sekaligus nilai saat iterasi list.

```
# Iterasi daftar mahasiswa - cek status
daftar_mhs = [
    {"nim": "2021001", "nama": "Andi", "ipk": 3.5, "sks": 20},
    {"nim": "2021002", "nama": "Budi", "ipk": 1.8, "sks": 15},
    {"nim": "2021003", "nama": "Cici", "ipk": 3.9, "sks": 22},
    {"nim": "2021004", "nama": "Dodi", "ipk": 2.5, "sks": 18},
]

print("=== Status Mahasiswa ===")
for mhs in daftar_mhs:
    if mhs["ipk"] >= 2.0 and mhs["sks"] >= 18:
        status = "Aktif"
    else:
        status = "Tidak Aktif"
    print(f"{mhs['nim']} {mhs['nama']:<15} IPK:{mhs['ipk']:.2f}
Status:{status}")

# Iterasi dengan enumerate untuk nomor urut
print("\n=== Daftar dengan Nomor ===")
for idx, mhs in enumerate(daftar_mhs, start=1):
    print(f"{idx}. {mhs['nama']} - IPK: {mhs['ipk']:.2f}")
```

5.2 For Loop dengan range()

Fungsi range() menghasilkan sequence bilangan yang digunakan untuk perulangan berdasarkan hitungan. range(stop) menghasilkan 0 sampai stop-1, range(start, stop) menghasilkan start sampai stop-1, dan range(start, stop, step) mengizinkan langkah kustom. Fungsi ini sangat berguna dalam konteks bisnis ketika perlu menghitung kumulatif (seperti total biaya beberapa semester), menghasilkan nomor urut, atau memproses data berdasarkan indeks.

```
# Hitung total biaya SPP 8 semester
spp_per_semester = 5000000
total_biaya = 0

for semester in range(1, 9):  # semester 1 sampai 8
    total_biaya += spp_per_semester
    print(f"Semester {semester}: Rp{spp_per_semester:,} | Total:
Rp{total_biaya:,}")

print(f"\nTotal biaya 8 semester: Rp{total_biaya:,}")

# Range dengan step - cek stok setiap 5 item
stok_barang = [45, 3, 28, 7, 12, 2, 50, 8, 15, 1]
batas_peringatan = 10

print("\n=== Cek Stok (setiap 5 item) ===")
for i in range(0, len(stok_barang), 5):
    stok = stok_barang[i]
    if stok <= batas_peringatan:
        print(f"Item {i}: STOK RENDAH ({stok})")
    else:
        print(f"Item {i}: Stok aman ({stok})")
```

5.3 While Loop: Perulangan Berdasarkan Kondisi

While loop melanjutkan perulangan selama kondisi bernilai True, membuatnya cocok untuk situasi di mana jumlah iterasi tidak diketahui sebelumnya. Dalam konteks Sistem Informasi, while loop digunakan untuk menu interaktif yang terus berjalan sampai pengguna memilih keluar, proses yang berulang sampai kondisi tertentu tercapai (misalnya input valid), atau simulasi yang berjalan sampai threshold terpenuhi. Perhatikan bahwa while loop rentan terhadap infinite loop jika kondisi tidak pernah menjadi False, sehingga pastikan ada mekanisme yang mengubah kondisi di dalam loop.

```
# Menu interaktif dengan while loop
print("=== Sistem Informasi Mahasiswa ===")

while True:
    print("\nMenu:")
    print("1. Cek Status Mahasiswa")
    print("2. Hitung Diskon")
    print("3. Cek Stok")
    print("0. Keluar")

    pilihan = input("Pilih menu: ")

    if pilihan == "1":
        ipk = float(input("  Masukkan IPK: "))
        status = "Aktif" if ipk >= 2.0 else "Tidak Aktif"
```

```
        print(f"  Status: {status}")
    elif pilihan == "2":
        biaya = float(input("  Biaya SPP: Rp"))
        diskon = 0.25 if input("  Anak karyawan? (y/n): ") == "y" else 0.0
        print(f"  Bayar: Rp{biaya * (1-diskon):,.0f}")
    elif pilihan == "3":
        stok = int(input("  Jumlah stok: "))
        if stok <= 10:
            print("  PERINGATAN: Stok rendah!")
        else:
            print("  Stok aman.")
    elif pilihan == "0":
        print("Terima kasih. Program selesai.")
        break
    else:
        print("Pilihan tidak valid!")
```

5.4 Break, Continue, dan Else pada Loop

Python menyediakan tiga kontrol alur tambahan dalam loop: break untuk menghentikan loop sepenuhnya, continue untuk melewati iterasi saat ini dan lanjut ke iterasi berikutnya, dan else yang dijalankan jika loop selesai tanpa break. Fitur else pada loop unik di Python dan sangat berguna untuk mendeteksi apakah loop selesai secara normal atau dihentikan oleh break, misalnya untuk mendeteksi apakah suatu item ditemukan dalam pencarian.

```
# Break: hentikan loop jika stok kritis ditemukan
stok_list = [45, 30, 2, 28, 15]  # item ke-3 kritis

for i, stok in enumerate(stok_list):
    if stok <= 3:
        print(f"STOK KRITIS di item {i}: {stok} unit!")
        break  # hentikan, tidak perlu cek lanjutan
    print(f"Item {i}: stok {stok} - aman")

# Continue: lewati mahasiswa cuti
daftar = [
    {"nama": "Andi", "is_cuti": False, "ipk": 3.5},
    {"nama": "Budi", "is_cuti": True, "ipk": 0.0},
    {"nama": "Cici", "is_cuti": False, "ipk": 3.8},
]

print("\nMahasiswa aktif saja:")
for mhs in daftar:
    if mhs["is_cuti"]:
        continue  # lewati yang cuti
    print(f"  {mhs['nama']} - IPK: {mhs['ipk']:.2f}")

# For-else: cek apakah semua stok aman
print("\nCek keamanan stok:")
```

```
for stok in stok_list:
    if stok <= 5:
        print("Ada stok yang tidak aman!")
        break
else:
    print("Semua stok aman.")  # hanya jika tanpa break
```

Bab 6: Validasi Input dan Aturan Bisnis

Validasi merupakan langkah kritis dalam implementasi aturan bisnis. Sebelum aturan bisnis dapat dievaluasi, data input harus dipastikan memenuhi syarat format dan rentang nilai yang diharapkan. Tanpa validasi, aturan bisnis mungkin menghasilkan keputusan yang salah berdasarkan data tidak valid, atau program mengalami error saat runtime. Dalam Sistem Informasi, validasi mencakup pemeriksaan format (NIM harus 10 digit, email harus mengandung @), rentang nilai (IPK antara 0.0 dan 4.0, SKS antara 0 dan 24), dan konsistensi logika (tanggal mulai sebelum tanggal selesai, SKS tempuh tidak melebihi SKS total).

6.1 Validasi Tipe dan Format

Validasi tipe memastikan data memiliki tipe yang sesuai sebelum diproses lebih lanjut. Validasi format memeriksa apakah data mengikuti pola yang ditentukan, seperti format NIM yang terdiri dari tahun masuk diikuti kode prodi. Teknik validasi dasar meliputi pengecekan tipe dengan isinstance(), pengecekan panjang dengan len(), pengecekan pola string dengan method string (isdigit(), isalpha(), startswith()), dan pengecekan keanggotaan dengan operator in. Untuk pola yang lebih kompleks, modul regex (re) menyediakan pencocokan pola yang sangat fleksibel.

```
# Validasi format NIM: 4 digit tahun + 2 digit prodi + 3 digit urut
def validasi_nim(nim):
    """Validasi format NIM mahasiswa."""
    if not isinstance(nim, str):
        return False, "NIM harus berupa string"
    if len(nim) != 10:
        return False, "NIM harus 10 karakter"
    if not nim.isdigit():
        return False, "NIM harus berisi angka saja"
    tahun = int(nim[:4])
    if tahun < 2000 or tahun > 2037:  # batas realistis
        return False, "Tahun masuk tidak valid"
    return True, "NIM valid"
```

```
# Validasi IPK: rentang 0.0 - 4.0
def validasi_ipk(ipk):
    """Validasi rentang IPK."""
    if not isinstance(ipk, (int, float)):
        return False, "IPK harus berupa angka"
    if ipk < 0.0 or ipk > 4.0:
        return False, "IPK harus antara 0.0 dan 4.0"
    return True, "IPK valid"

# Penggunaan
nim_test = "2021001234"
valid, msg = validasi_nim(nim_test)
print(f"NIM {nim_test}: {msg}")  # NIM 10 digit: valid

ipk_test = 4.5
valid, msg = validasi_ipk(ipk_test)
print(f"IPK {ipk_test}: {msg}")  # IPK harus antara 0.0 dan 4.0
```

6.2 Validasi dengan Loop: Input Sampai Valid

Pola umum dalam aplikasi interaktif adalah meminta input berulang kali sampai pengguna memasukkan data yang valid. Pola ini menggabungkan while loop dengan validasi, di mana loop terus berjalan selama input tidak valid dan baru berhenti (break) ketika validasi lulus. Pendekatan ini memberikan pengalaman pengguna yang lebih baik karena program tidak crash akibat input tidak valid, melainkan memberikan pesan error spesifik dan meminta ulang input.

```
# Pola input-validasi dengan while loop

# Validasi IPK - ulangi sampai valid
while True:
    try:
        ipk = float(input("Masukkan IPK (0.0-4.0): "))
        if 0.0 <= ipk <= 4.0:
            break  # input valid, keluar loop
        else:
            print("  Error: IPK harus antara 0.0 dan 4.0")
    except ValueError:
        print("  Error: Masukkan angka yang valid")

print(f"IPK yang dimasukkan: {ipk:.2f}")

# Validasi pilihan menu - ulangi sampai valid
menu_valid = ["1", "2", "3", "0"]
while True:
    pilihan = input("Pilih menu (1/2/3/0): ")
    if pilihan in menu_valid:
```

```
        break
    print(f"  Error: Pilih dari {menu_valid}")

print(f"Menu dipilih: {pilihan}")
```

6.3 Validasi Aturan Bisnis Komposit

Aturan bisnis seringkali melibatkan validasi komposit, yaitu kombinasi beberapa kondisi yang harus dipenuhi secara bersamaan. Misalnya, persetujuan cuti akademik mungkin memerlukan: (1) IPK di atas batas tertentu, (2) tidak memiliki tunggakan biaya, (3) sudah menempuh minimal 2 semester, dan (4) tidak sedang dalam proses skripsi. Implementasi validasi komposit yang terstruktur memudahkan pemeliharaan dan penambahan aturan baru, serta menghasilkan pesan error yang spesifik untuk setiap kondisi yang tidak terpenuhi.

```
# Validasi komposit: Persetujuan cuti akademik
def cek_pengajuan_cuti(mahasiswa):
    """Cek kelayakan pengajuan cuti akademik."""
    error_list = []

    # Kondisi 1: IPK minimum
    if mahasiswa["ipk"] < 1.5:
        error_list.append("IPK di bawah 1.5 (risiko drop out)")

    # Kondisi 2: Tidak ada tunggakan
    if mahasiswa["tunggakan"] > 0:
        error_list.append(f"Tunggakan Rp{mahasiswa['tunggakan']:,} belum
lunas")

    # Kondisi 3: Minimal semester 2
    if mahasiswa["semester"] < 2:
        error_list.append("Belum menempuh semester 2")

    # Kondisi 4: Tidak sedang skripsi
    if mahasiswa["is_skripsi"]:
        error_list.append("Sedang dalam proses skripsi")

    # Hasil validasi
    if len(error_list) == 0:
        return True, "Pengajuan cuti dapat diproses"
    else:
        return False, "Alasan: " + "; ".join(error_list)

# Contoh penggunaan
mhs = {
    "nim": "2021001", "ipk": 3.2, "semester": 4,
    "tunggakan": 500000, "is_skripsi": False
}
```

```
layak, pesan = cek_pengajuan_cuti(mhs)
print(f"Layak cuti: {layak}")
print(f"Pesan: {pesan}")
```

Bab 7: Studi Kasus Integratif

Bab ini mengintegrasikan seluruh konstruksi yang telah dipelajari, yaitu variabel, I/O, kondisional, perulangan, dan validasi, ke dalam tiga studi kasus lengkap yang merepresentasikan aturan bisnis nyata dalam Sistem Informasi. Setiap studi kasus mencakup analisis aturan bisnis, desain algoritma, implementasi kode, dan contoh pengujian. Ketiga studi kasus tersebut adalah: (1) Sistem Penentuan Status Mahasiswa, (2) Kalkulator Diskon Biaya Kuliah, dan (3) Sistem Peringatan Stok Barang.

7.1 Studi Kasus 1: Sistem Status Mahasiswa

Sistem penentuan status mahasiswa mengimplementasikan aturan bisnis kampus terkait klasifikasi status akademik. Aturan bisnis yang diimplementasikan: mahasiswa berstatus Aktif jika IPK >= 2.0 dan SKS >= 18 dan tidak cuti; berstatus Peringatan jika IPK antara 1.5 dan 2.0; berstatus Tidak Aktif jika IPK < 1.5; dan berstatus Cuti jika mengajukan cuti. Program menerima data beberapa mahasiswa dan menghasilkan laporan status seluruh mahasiswa beserta rekomendasi tindakan.

```
# studi_kasus_status_mahasiswa.py
"""Sistem Penentuan Status Mahasiswa."""

# Konstanta aturan bisnis
IPK_MIN_AKTIF = 2.0
IPK_MIN_PERINGATAN = 1.5
SKS_MIN = 18

def tentukan_status(mhs):
    """Tentukan status akademik mahasiswa."""
    if mhs["is_cuti"]:
        return "Cuti", "Lakukan administrasi cuti"

    ipk = mhs["ipk"]
    sks = mhs["sks"]

    if ipk >= IPK_MIN_AKTIF and sks >= SKS_MIN:
        return "Aktif", "Lanjutkan studi semester berikutnya"
    elif ipk >= IPK_MIN_PERINGATAN:
        if sks < SKS_MIN:
```

```
            return "Peringatan", "IPK memadai, tapi SKS kurang"
        return "Peringatan", "IPK di bawah batas, konsultasi dosen PA"
    else:
        return "Tidak Aktif", "Evaluasi lanjut: risiko drop out"

# Data mahasiswa (simulasi)
daftar_mhs = [
    {"nim": "2021001", "nama": "Andi", "ipk": 3.52, "sks": 20, "is_cuti":
False},
    {"nim": "2021002", "nama": "Budi", "ipk": 1.75, "sks": 15, "is_cuti":
False},
    {"nim": "2021003", "nama": "Cici", "ipk": 3.90, "sks": 22, "is_cuti":
False},
    {"nim": "2021004", "nama": "Dodi", "ipk": 1.20, "sks": 12, "is_cuti":
False},
    {"nim": "2021005", "nama": "Eka", "ipk": 3.00, "sks": 18, "is_cuti": True},
]

# Proses dan tampilkan laporan
print("=" * 60)
print("LAPORAN STATUS MAHASISWA")
print("=" * 60)
print(f"{'NIM':<10} {'Nama':<10} {'IPK':>5} {'SKS':>4} {'Status':<14}
Rekomendasi")
print("-" * 60)

for mhs in daftar_mhs:
    status, rekomendasi = tentukan_status(mhs)
    print(f"{mhs['nim']:<10} {mhs['nama']:<10} {mhs['ipk']:>5.2f}
{mhs['sks']:>4} {status:<14} {rekomendasi}")
```

7.2 Studi Kasus 2: Diskon Biaya Kuliah

Kalkulator diskon biaya kuliah mengimplementasikan aturan bisnis keuangan kampus. Aturan yang diimplementasikan: diskon 25% untuk anak karyawan internal, diskon 20% untuk IPK >= 3.8 (prestasi tinggi), diskon 15% untuk IPK >= 3.5 (prestasi baik), diskon 10% untuk IPK >= 3.0 (memuaskan), dan diskon tambahan 5% untuk pembayaran tepat waktu. Diskon tidak kumulatif; hanya diskon terbesar yang diberikan, kecuali diskon tepat waktu yang selalu ditambahkan. Program menerima data mahasiswa dan biaya SPP, lalu menghitung biaya final setelah diskon.

```
# studi_kasus_diskon_biaya.py
"""Kalkulator Diskon Biaya Kuliah."""

# Konstanta aturan bisnis
DISKON_ANAK_KARYAWAN = 0.25
DISKON_PRESTASI_TINGGI = 0.20   # IPK >= 3.8
DISKON_PRESTASI_BAIK = 0.15    # IPK >= 3.5
```

```
DISKON_MEMUASKAN = 0.10        # IPK >= 3.0
DISKON_TEPAT_WAKTU = 0.05      # pembayaran tepat waktu

def hitung_diskon(ipk, is_anak_karyawan, is_tepat_waktu):
    """Hitung diskon berdasarkan aturan bisnis."""
    # Diskon utama: ambil yang terbesar
    if is_anak_karyawan:
        diskon_utama = DISKON_ANAK_KARYAWAN
        kategori = "Anak Karyawan"
    elif ipk >= 3.8:
        diskon_utama = DISKON_PRESTASI_TINGGI
        kategori = "Prestasi Tinggi"
    elif ipk >= 3.5:
        diskon_utama = DISKON_PRESTASI_BAIK
        kategori = "Prestasi Baik"
    elif ipk >= 3.0:
        diskon_utama = DISKON_MEMUASKAN
        kategori = "IPK Memuaskan"
    else:
        diskon_utama = 0.0
        kategori = "Tidak ada diskon"

    # Diskon tambahan: tepat waktu (selalu ditambahkan)
    diskon_tambahan = DISKON_TEPAT_WAKTU if is_tepat_waktu else 0.0
    total_diskon = diskon_utama + diskon_tambahan

    return total_diskon, kategori, diskon_utama, diskon_tambahan

# Simulasi perhitungan
biaya_spp = 5000000
test_cases = [
    (3.9, False, True, "Cici - Prestasi + Tepat Waktu"),
    (3.5, True, False, "Andi - Anak Karyawan"),
    (2.8, False, True, "Budi - Tepat Waktu saja"),
    (3.2, False, False, "Dodi - Memuaskan saja"),
]

print("=== Simulasi Diskon Biaya Kuliah ===")
print(f"SPP Dasar: Rp{biaya_spp:,}\n")

for ipk, is_ak, is_tw, deskripsi in test_cases:
    total_d, kategori, d_utama, d_tambahan = hitung_diskon(ipk, is_ak, is_tw)
    potongan = biaya_spp * total_d
    bayar = biaya_spp - potongan
    print(f"{deskripsi}")
    print(f"  Kategori: {kategori} | Diskon: {total_d:.0%} | Bayar:
Rp{bayar:,}")
```

7.3 Studi Kasus 3: Sistem Peringatan Stok

Sistem peringatan stok mengimplementasikan aturan bisnis manajemen inventaris. Aturan yang diimplementasikan: stok di atas 20 unit berstatus

Aman, stok 11-20 berstatus Peringatan (perlu rencana restock), stok 1-10 berstatus Rendah (restock segera), dan stok 0 berstatus Habis (prioritas utama). Program menerima data inventaris, mengevaluasi status setiap item, menghitung total nilai inventaris, dan menghasilkan laporan yang mencakup ringkasan status dan rekomendasi tindakan.

```
# studi_kasus_peringatan_stok.py
"""Sistem Peringatan Stok Barang."""

# Konstanta aturan bisnis
BATAS_AMAN = 20
BATAS_PERINGATAN = 10

def cek_status_stok(stok):
    """Tentukan status dan rekomendasi berdasarkan stok."""
    if stok == 0:
        return "HABIS", "URGENT: Restock segera!", "CRITICAL"
    elif stok <= BATAS_PERINGATAN:
        return "RENDAH", "Restock dalam 1-3 hari", "HIGH"
    elif stok <= BATAS_AMAN:
        return "PERINGATAN", "Rencanakan restock 1-2 minggu", "MEDIUM"
    else:
        return "AMAN", "Stok mencukupi", "LOW"

# Data inventaris
inventaris = [
    {"kode": "B001", "nama": "Buku Python", "stok": 45, "harga": 85000},
    {"kode": "B002", "nama": "Buku Database", "stok": 12, "harga": 95000},
    {"kode": "B003", "nama": "Flashdisk 16GB", "stok": 5, "harga": 50000},
    {"kode": "B004", "nama": "Kabel USB", "stok": 0, "harga": 25000},
    {"kode": "B005", "nama": "Mouse Wireless", "stok": 18, "harga": 120000},
]

# Proses dan laporan
print("=" * 70)
print("LAPORAN STATUS INVENTARIS")
print("=" * 70)

total_nilai = 0
count_peringatan = 0

for item in inventaris:
    status, rekomendasi, prioritas = cek_status_stok(item["stok"])
    nilai = item["stok"] * item["harga"]
    total_nilai += nilai

    if prioritas in ["HIGH", "CRITICAL"]:
        count_peringatan += 1

    print(f"{item['kode']} {item['nama']:<15} Stok:{item['stok']:>3} "
          f"Status:{status:<12} [{prioritas}]")
```

```
    if prioritas != "LOW":
        print(f"  -> {rekomendasi}")

print(f"\nTotal Nilai Inventaris: Rp{total_nilai:,}")
print(f"Item Perlu Tindakan: {count_peringatan} dari {len(inventaris)}")
```

Rangkuman

Modul praktikum ini telah membahas implementasi aturan bisnis menggunakan konstruksi dasar Python. Variabel dan tipe data menyediakan fondasi untuk merepresentasikan entitas dan state bisnis, dengan penekanan pada konvensi penamaan yang deskriptif dan pemilihan tipe data yang tepat untuk menghindari bug halus. Mekanisme input/output memungkinkan program berinteraksi dengan pengguna, di mana input() menerima data sebagai string yang perlu dikonversi dan divalidasi, sedangkan print() dengan f-string menyajikan output yang terformat profesional.

Struktur kondisional (if-elif-else) merupakan alat utama mengimplementasikan keputusan bisnis, mulai dari percabangan biner sederhana hingga percabangan berantai untuk kategorisasi multi-level dan percabangan bersarang untuk aturan bertingkat. Struktur perulangan (for dan while) memproses data secara iteratif, dengan for loop untuk iterasi over sequence dan while loop untuk perulangan berbasis kondisi termasuk menu interaktif. Validasi input memastikan data memenuhi syarat sebelum aturan bisnis dievaluasi, meliputi validasi tipe, format, rentang, dan kondisi komposit.

Tiga studi kasus integratif telah mendemonstrasikan penerapan seluruh konstruksi secara bersamaan: Sistem Status Mahasiswa menggabungkan kondisional dan perulangan untuk klasifikasi akademik, Kalkulator Diskon Biaya mengimplementasikan aturan keuangan dengan prioritas diskon, dan Sistem Peringatan Stok menerapkan kategorisasi level dengan rekomendasi tindakan. Keterampilan yang diperoleh dalam modul ini merupakan fondasi esensial untuk pengembangan Sistem Informasi yang lebih kompleks pada modul-modul selanjutnya.

Latihan Soal

A. Soal Teori

1. Jelaskan perbedaan antara aturan bisnis (business rules) dan logika teknis (technical logic) dalam konteks pengembangan Sistem Informasi. Berikan masing-masing dua contoh dari Sistem Informasi Mahasiswa.
2. Mengapa NIM mahasiswa sebaiknya disimpan sebagai string (str) dan bukan integer (int)? Jelaskan konsekuensi yang terjadi jika NIM disimpan sebagai integer.
3. Apa perbedaan antara operator logika and dan or dalam konteks evaluasi aturan bisnis? Berikan contoh aturan bisnis yang menggunakan masing-masing operator.
4. Jelaskan mengapa urutan kondisi dalam if-elif-else penting. Apa yang terjadi jika kondisi yang lebih umum ditempatkan sebelum kondisi yang lebih spesifik?
5. Kapan sebaiknya menggunakan for loop dan kapan menggunakan while loop? Berikan contoh skenario bisnis untuk masing-masing.
6. Apa fungsi klausa else pada for loop dan while loop di Python? Bagaimana perilakunya berbeda dari else pada if-else?
7. Mengapa validasi input harus dilakukan sebelum mengevaluasi aturan bisnis? Jelaskan risiko yang terjadi jika validasi diabaikan.
8. Jelaskan konsep validasi komposit dan berikan contoh implementasinya untuk aturan bisnis pengajuan beasiswa.

B. Soal Praktik

1. Buatlah program Python yang menerima data 5 mahasiswa (NIM, nama, IPK, SKS) menggunakan loop, lalu menampilkan laporan status setiap mahasiswa (Aktif/Peringatan/Tidak Aktif) berdasarkan aturan: Aktif jika IPK >= 2.0 dan SKS >= 18, Peringatan jika 1.5 <= IPK < 2.0, dan Tidak Aktif jika IPK < 1.5. Simpan sebagai status_mahasiswa.py.
2. Buatlah program kalkulator diskon biaya kuliah yang menerima input biaya SPP, status anak karyawan (y/n), IPK, dan status pembayaran

tepat waktu (y/n). Program menghitung dan menampilkan diskon yang berlaku, potongan biaya, dan biaya final. Gunakan aturan diskon: anak karyawan 25%, IPK >= 3.8 diskon 20%, IPK >= 3.5 diskon 15%, IPK >= 3.0 diskon 10%, dan tepat waktu tambahan 5%. Simpan sebagai diskon_biaya.py.

3. Buatlah program peringatan stok yang menggunakan data inventaris berupa list of dictionary (minimal 8 item dengan variasi stok: aman, peringatan, rendah, habis). Program mengevaluasi status setiap item, menampilkan laporan lengkap dengan prioritas, menghitung total nilai inventaris, dan menampilkan ringkasan statistik. Simpan sebagai peringatan_stok.py.
4. Buatlah program dengan menu interaktif (while loop) yang menggabungkan ketiga studi kasus: (1) Cek Status Mahasiswa, (2) Hitung Diskon Biaya, (3) Cek Peringatan Stok, dan (0) Keluar. Setiap submenu harus berfungsi penuh dengan validasi input. Simpan sebagai menu_si_mahasiswa.py.
5. Buatlah fungsi validasi untuk data mahasiswa yang mencakup: validasi NIM (10 digit angka, tahun 2000-2037), validasi IPK (0.0-4.0), validasi SKS (0-24), dan validasi semester (1-14). Fungsi mengembalikan list error jika ada data tidak valid. Tulis program yang menerima input dan menampilkan hasil validasi. Simpan sebagai validasi_data.py.
6. Buatlah program yang menghitung ringkasan algoritma dari ketiga studi kasus: (a) Hitung persentase mahasiswa aktif vs tidak aktif, (b) Hitung rata-rata diskon yang diberikan, (c) Hitung persentase item yang perlu restock. Tampilkan hasil dalam format tabel. Simpan sebagai ringkasan_algoritma.py.